In [ ]:
```
#%pip install -q pandas scikit-learn
```

In [1]:
```
import re
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

CSV_PATH = "twitter_sentiment.csv"  # Change this
TEXT_COLUMN = "text"                # Change this
LABEL_COLUMN = "sentiment"          # Change this

df = pd.read_csv(r"C:\Users\om_jangam\Downloads\twitter_sentiment.csv")
df = df[[TEXT_COLUMN, LABEL_COLUMN]].dropna().drop_duplicates()
df[TEXT_COLUMN] = df[TEXT_COLUMN].astype(str)
df[LABEL_COLUMN] = df[LABEL_COLUMN].astype(str).str.lower().str.strip()
print(df.shape)
display(df.head())
print(df[LABEL_COLUMN].value_counts())
```

```
(200, 2)
```

	text	sentiment
0	I love this product, it works perfectly	positive
1	I love this product, it works perfectly!	positive
2	The service was excellent and very helpful	positive
3	The service was excellent and very helpful!	positive
4	Had a wonderful day with my friends	positive

```
sentiment
positive    100
negative    100
Name: count, dtype: int64
```

In [2]:
```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " ", text)
    text = re.sub(r"@\w+", " ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

df["clean_text"] = df[TEXT_COLUMN].apply(clean_text)

# Keep only positive and negative classes (edit aliases if your dataset uses other
label_map = {
    "positive": "positive", "pos": "positive", "1": "positive",
    "negative": "negative", "neg": "negative", "0": "negative"
}
df["sentiment_clean"] = df[LABEL_COLUMN].map(label_map)
df = df.dropna(subset=["sentiment_clean"])
```

```
if df["sentiment_clean"].nunique() != 2:
    raise ValueError("Need both positive and negative labels. Check LABEL_COLUMN an

X_train_text, X_test_text, y_train, y_test = train_test_split(
    df["clean_text"], df["sentiment_clean"], test_size=0.2,
    random_state=42, stratify=df["sentiment_clean"]
)
vectorizer = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)
print("Training shape:", X_train.shape, "Testing shape:", X_test.shape)
```

```
Training shape: (160, 570) Testing shape: (40, 570)
```

In [3]:
```
# Cosine distance is often useful for sparse TF-IDF vectors
knn = KNeighborsClassifier(n_neighbors=5, metric="cosine", algorithm="brute")
knn.fit(X_train, y_train)
predictions = knn.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, predictions), 4))
print("\nConfusion matrix:\n", confusion_matrix(y_test, predictions, labels=["negat
print("\nClassification report:\n",
      classification_report(y_test, predictions, zero_division=0))
```

```
Accuracy: 0.55

Confusion matrix:
 [[12  8]
 [10 10]]

Classification report:
              precision    recall  f1-score   support

    negative       0.55      0.60      0.57        20
    positive       0.56      0.50      0.53        20

    accuracy                           0.55        40
   macro avg       0.55      0.55      0.55        40
weighted avg       0.55      0.55      0.55        40
```

In [4]:
```
# Test the classifier on your own tweets
sample_tweets = [
    "I absolutely love this product, it is wonderful",
    "This is a terrible experience and I hate it"
]
sample_features = vectorizer.transform([clean_text(t) for t in sample_tweets])
sample_predictions = knn.predict(sample_features)
for tweet, sentiment in zip(sample_tweets, sample_predictions):
    print(f"{tweet} -> {sentiment}")
```

```
I absolutely love this product, it is wonderful -> positive
This is a terrible experience and I hate it -> negative
```